



CLASE 4 • 3 HS

Funciones y Tablas de Verdad



Dejar de copiar y pegar: bucles y funciones para que el ESP32 imprima **cualquier tabla de verdad**.

HOY

Dos herramientas: bucles y funciones

1 • 30 min

El bucle for

2 • 20 min

La lista de pines

3 • 50 min

Funciones

4 • 35 min

Tablas de verdad

5 • 45 min

Ejercicios

TP

**El entrenador imprime
tablas**



Promesa cumplida: el barrido en 5 líneas

```
for (int i = 0; i < 8; i++) {  
    digitalWrite(LEDs[i], HIGH);  
    delay(150);  
    digitalWrite(LEDs[i], LOW);  
}
```

16

líneas en la Clase 1, para 4 LEDs

5

líneas hoy, para los 8 LEDs

Al final de la clase se entiende cada carácter. Y con las mismas dos herramientas, **bucles y funciones**, se generan tablas de verdad sin equivocarse nunca en una fila.



El for: tres partes entre paréntesis

```
for (int i = 0; i < 4; i++) {  
    Serial.printf("Vuelta %d\n", i);  
}  
Serial.printf("Listo\n");
```

Vuelta 0
Vuelta 1
Vuelta 2
Vuelta 3
Listo

Parte	Qué hace	Cuándo
inicio <code>int i = 0</code>	Crea el contador y le da su primer valor	Una sola vez, al entrar
condición <code>i < 4</code>	Verdadera: ejecuta el cuerpo. Falsa: sale	Antes de cada vuelta
paso <code>i++</code>	Suma 1 al contador (<code>i = i + 1</code>)	Después de cada vuelta



Seguir el for a mano, en el cuaderno

Momento	i	¿i < 4?	¿Qué pasa?
inicio	0	sí	imprime Vuelta 0
paso	1	sí	imprime Vuelta 1
paso	2	sí	imprime Vuelta 2
paso	3	sí	imprime Vuelta 3
paso	4	no	sale del for → imprime Listo

$i < N$ repite N veces ($i = 0 \dots N-1$). Para que el último entre: $n \leq 15$ ($n = 0 \dots 15$).



Nunca ; después del paréntesis: **for (...);** repite “nada” y el cuerpo corre **una sola vez**. Compila sin error. Si usa i: *'i' was not declared in this scope.*



El otro bucle: while, “mientras”

```
int potencia = 1;
while (potencia < 100) {
    Serial.printf("%d ", potencia);
    potencia = potencia * 2;
}
// imprime: 1 2 4 8 16 32 64
```

Bucle infinito

Si la condición nunca se vuelve falsa, el programa queda trabado ahí **para siempre**. Sin la línea que duplica, potencia vale 1 eternamente.

Usar...	cuando...
for	se sabe cuántas veces repetir (8 LEDs, 16 números, 4 filas)
while	se repite hasta que pase algo (un botón, llegar a un valor)



Un arreglo: la lista de pines

```
const int LEDS[8] = {4, 16, 17, 18, 19, 21, 22, 23}; // L0 ... L7
```

Índice i	0	1	2	3	4	5	6	7
LEDS[i]	4	16	17	18	19	21	22	23
LED	L0	L1	L2	L3	L4	L5	L6	L7

 Los índices **empiezan en 0**, como los bits: LEDS[i] es el LED del **bit i**.

 **LEDS[8] no existe.** Arreglo de 8 → **i < 8**, nunca i <= 8.



Una función: ponerle nombre a una idea

```
void imprimirSeparador() {  
    Serial.printf("-----\n");  
}
```

Ya las usamos: **pinMode**, **delay**, **Serial.printf**. Y **setup()** y **loop()** son funciones propias.

Parte	Qué significa
void	La función no devuelve ningún resultado (“vacío”)
imprimirSeparador	El nombre: que diga qué hace
()	Los parámetros (aquí, ninguno)
{ ... }	El cuerpo : se ejecuta cada vez que se la llama
imprimirSeparador();	La llamada : nombre, paréntesis y punto y coma



Datos que entran, resultado que sale

```
void parpadear(int pin, int veces) {  
    for (int i = 0; i < veces; i++) {  
        digitalWrite(pin, HIGH);  
        delay(150);  
        digitalWrite(pin, LOW);  
        delay(150);  
    }  
}  
parpadear(LED0, 3); // 3 veces
```

```
int cuadrado(int x) {  
    int resultado = x * x;  
    return resultado;  
}  
  
int c = cuadrado(5);           // c vale 25  
Serial.printf("%d\n", cuadrado(7)); // 49
```

Los valores se copian **en orden**. **return** termina la función; si dice int o bool y falta el return, devuelve cualquier cosa.



Variables locales: solo viven adentro

```
int cuadrado(int x) {  
    int resultado = x * x; // solo existe aquí  
    return resultado;  
}  
  
void setup() {  
    Serial.printf("%d\n", resultado); // ERROR  
}
```

'resultado' was not declared in this scope

Nacen cuando la función arranca y **desaparecen** cuando termina. Por eso se puede usar **i** en diez funciones sin que se pisen.

Los parámetros son **copias**. Lo declarado **afuera** de toda función, como LEDS o BOTON_A, es **global**: lo ve todo el programa.



Definir antes de usar: el prototipo

```
void mostrarNumero(int n); // prototipo

void setup() {
  mostrarNumero(5); // ya la conoce
}

void mostrarNumero(int n) {
  // la definición, más abajo
}
```

PlatformIO • main.cpp

Obligatorio. Si no: *'mostrarNumero' was not declared in this scope*

Wokwi • sketch.ino

A veces compila igual: Arduino genera los prototipos solo. En C de verdad, no.

El compilador lee **de arriba hacia abajo**. La otra salida: escribir las funciones **arriba** de setup() y loop().



Una compuerta es una función

$S = f(A, B)$



`bool f(bool a, bool b)`

```
bool compuertaAND(bool a, bool b) {  
    return a && b;  
}  
bool compuertaXOR(bool a, bool b) {  
    return a != b;  
}  
bool s = compuertaAND(1, 0); // s vale 0
```

Entradas binarias, una salida binaria: para cada combinación, **un único valor**. Un circuito combinatorial es una función que llama a otras.



mostrarNumero: binario en los LEDs

```
void mostrarNumero(int n) {
  for (int i = 0; i < 4; i++) {
    digitalWrite(LEDs[i], n % 2);
    n = n / 2;
  }
}
```

i	n entra	n % 2 → LED	n / 2
0	6	0 → L0 apagado	3
1	3	1 → L1 encendido	1
2	1	1 → L2 encendido	0
3	0	0 → L3 apagado	0

💡 L3 L2 L1 L0 = **0 1 1 0** = 6. Es el método de **divisiones sucesivas** de la teoría: cada resto va directo a su LED.



Dos entradas: dos for anidados

```
Serial.printf(" A | B | S\n");
Serial.printf("---+---+---\n");
for (int a = 0; a ≤ 1; a++) {
    for (int b = 0; b ≤ 1; b++) {
        bool s = compuertaAND(a, b);
        Serial.printf(" %d | %d | %d\n", a, b, s);
    }
}
```

A		B		S
0		0		0
0		1		0
1		0		0
1		1		1



Como un cuentakilómetros: **B gira completo** antes de que A avance uno. $2 \times 2 = 4$ filas, en el orden del pizarrón: 00, 01, 10, 11.



Tres entradas: un solo for de 0 a 7

$F = A \cdot B + C'$



```
return (a && b) || !c;
```

```
for (int fila = 0; fila < 8; fila++) {  
    int a = (fila / 4) % 2; // peso 4  
    int b = (fila / 2) % 2; // peso 2  
    int c = fila % 2;       // peso 1  
    Serial.printf(" %d | %d | %d | %d\n",  
        a, b, c, funcionF(a, b, c));  
}
```

Fila = mintérmino

La fila 5 (A=1, B=0, C=1) es $m5 = A \cdot B' \cdot C$. Donde la salida vale 1, ese mintérmino entra en la **suma de productos** canónica.



Ahora ustedes

EJERCICIO 1

Mayoría de 3

mayoria(a, b, c) vale 1 si al menos dos entradas valen 1.
Tabla de 8 filas con tres for anidados.

EJERCICIO 2

Verificar De Morgan

$\neg(a \ \&\& \ b)$ contra $\neg a \ || \ \neg b$ en todas las combinaciones: OK o ERROR. Igual con la segunda ley.

EJERCICIO 3

potencia2

potencia2(n) devuelve 2^n con un for. Imprimir el peso de cada LED: L0 pesa 1 ... L7 pesa 128.

EJERCICIO 4

Mintérminos

Un solo for por las 8 filas e imprimir $F = \sum m(3,5,6,7)$. Ojo con las comas.



El entrenador imprime tablas de verdad

1. Encabezado y pinout con el arreglo LEDS[8], BOTON_A y BOTON_B; los 8 LEDs se configuran con un for.
2. Una función por compuerta: AND, OR, XOR, NAND, NOR y XNOR.
3. evaluar(compuerta, a, b): 0 AND · 1 OR · 2 XOR · 3 NAND · 4 NOR · 5 XNOR · 6 A · 7 B.
4. imprimirTabla(compuerta); en setup(), con un for, las tablas de AND, OR, XOR y NAND.
5. La tabla de $F = A \cdot B + C'$ con un solo for de 0 a 7.
6. En loop(), el modo compuertas del TP3 con dos for: evaluar(i, a, b) en LEDS[i] e imprimirNombre(i).

```
=====
Entrenador Logico de Ana
Sistemas Logicos - UNSE 2026
Clase 4: tablas de verdad
=====
```

Tabla de AND

A	B	S
0	0	0

Entrega: carpeta **SL2026-Apellido-Clase4** en ZIP, sin la carpeta .pio.



Al terminar la clase, sé...

- ✓ explicar las tres partes de un for y seguirlo a mano en una tabla
- ✓ elegir entre for y while según el problema
- ✓ recorrer los 8 LEDs con LEDS[i] y explicar por qué i va de 0 a 7
- ✓ escribir y llamar una función con parámetros y otra con return
- ✓ explicar qué es una variable local
- ✓ generar la tabla de verdad de una función de 2 o 3 variables

La semana que viene: hoy sacamos los bits dividiendo por 2. En la Clase 5, C tiene **operadores para tocar los bits directamente** y los 8 LEDs se vuelven un byte.

